

From Data to Solutions · Weekend 7



Counting the FLOPs of a transformer

Where the 6 in 6ND comes from

Carlos Cotrini



Section 1

Where the 6 comes from



Where the 6 comes from

At 08:00 we used the 6 without deriving it

$$C = 6 N D$$

Where the 6 comes from

At 08:00 we used the 6 without deriving it

$$C = 6 N D$$

$$N = 70 \times 10^9 \text{ parameters}$$

At 08:00 we used the 6 without deriving it

$$C = 6 N D$$

$$N = 70 \times 10^9 \text{ parameters}$$

$$D = 15 \times 10^{12} \text{ training tokens}$$

At 08:00 we used the 6 without deriving it

$$C = 6 N D$$

$$N = 70 \times 10^9 \text{ parameters}$$

$$D = 15 \times 10^{12} \text{ training tokens}$$

$$C = 6.30 \times 10^{24} \text{ FLOPs}$$

Where the 6 comes from

At 08:00 we used the 6 without deriving it

The one number we did not derive

$$C = \boxed{6} N D$$

$$N = 70 \times 10^9 \text{ parameters}$$

$$D = 15 \times 10^{12} \text{ training tokens}$$

$$C = 6.30 \times 10^{24} \text{ FLOPs}$$

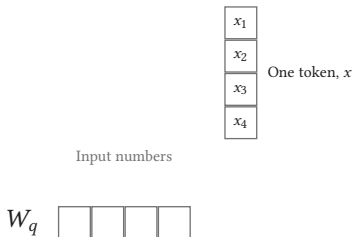
One query number: four multiplies, four adds

Input numbers

W_q

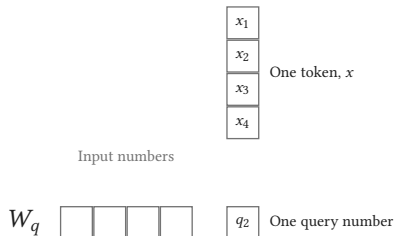
--	--	--	--

One query number: four multiplies, four adds



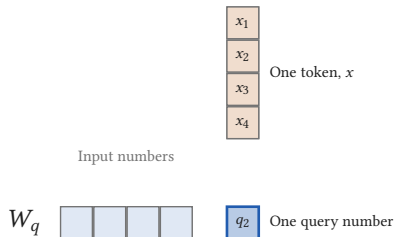
- A token is 4 numbers

One query number: four multiplies, four adds



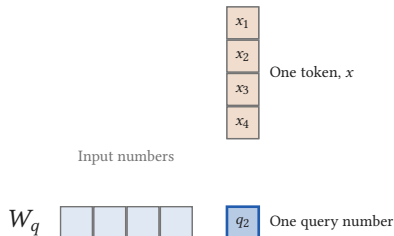
- A token is 4 numbers
- One row of W_q is 4 weights, and it makes ONE query number

One query number: four multiplies, four adds



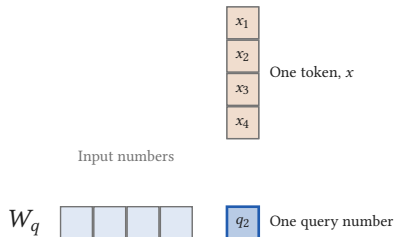
- A token is 4 numbers
- One row of W_q is 4 weights, and it makes ONE query number
- Multiply each weight by its number: 4 multiplies

One query number: four multiplies, four adds



- A token is 4 numbers
- One row of W_q is 4 weights, and it makes ONE query number
- Multiply each weight by its number: 4 multiplies
- Add the four products: 4 adds

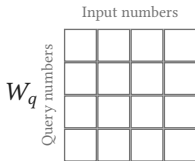
One query number: four multiplies, four adds



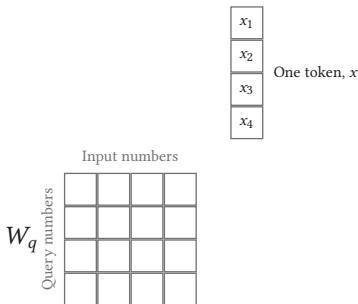
- A token is 4 numbers
- One row of W_q is 4 weights, and it makes ONE query number
- Multiply each weight by its number: 4 multiplies
- Add the four products: 4 adds
- 8 FLOP for 4 parameters, so 2 FLOP per parameter

Where the 6 comes from

Sixteen weights, sixteen multiply accumulates

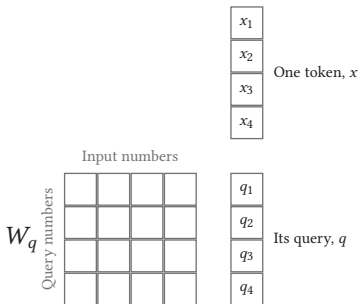


Sixteen weights, sixteen multiply accumulates



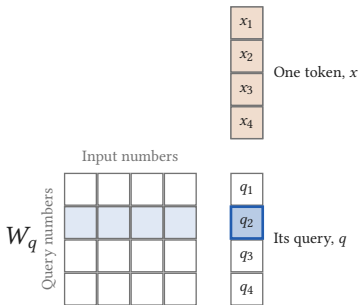
- A token is 4 numbers, and its query is 4 numbers

Sixteen weights, sixteen multiply accumulates



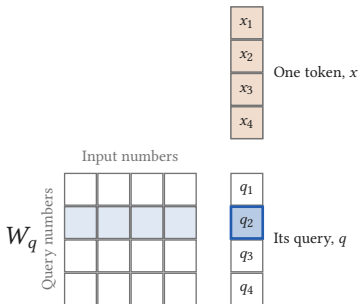
- A token is 4 numbers, and its query is 4 numbers

Sixteen weights, sixteen multiply accumulates



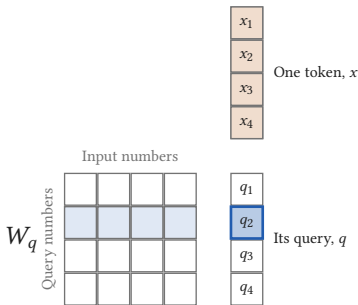
- A token is 4 numbers, and its query is 4 numbers
- One query number: 4 multiplies and 4 adds

Sixteen weights, sixteen multiply accumulates



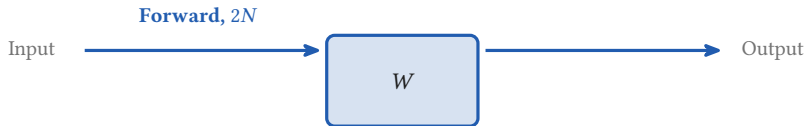
- A token is 4 numbers, and its query is 4 numbers
- One query number: 4 multiplies and 4 adds
- All four: 16 multiply accumulates, 32 FLOP

Sixteen weights, sixteen multiply accumulates

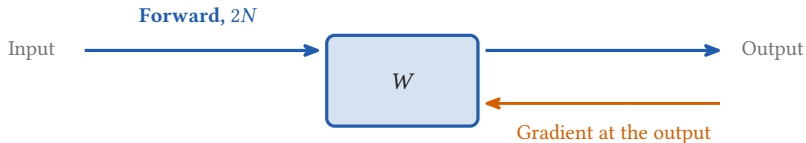


- A token is 4 numbers, and its query is 4 numbers
- One query number: 4 multiplies and 4 adds
- All four: 16 multiply accumulates, 32 FLOP
- Two FLOPs per parameter, per token, so the forward pass is $2N$

The backward pass computes two gradients

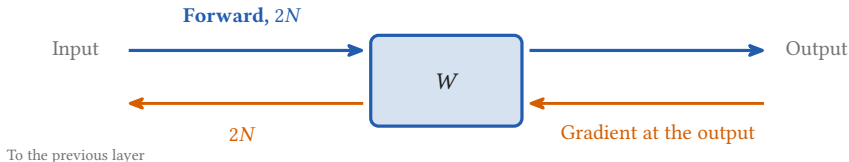


The backward pass computes two gradients



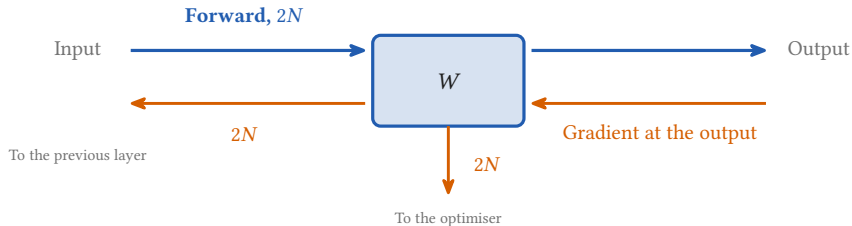
- The loss comes back as a gradient at the output

The backward pass computes two gradients



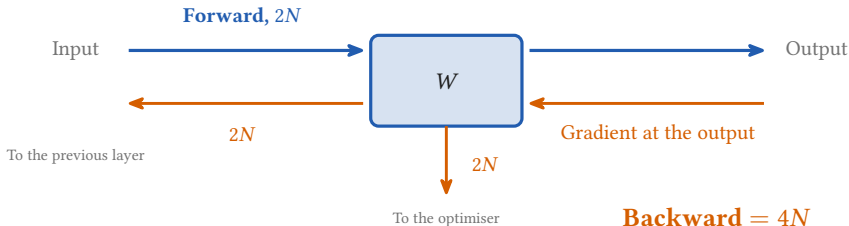
- The loss comes back as a gradient at the output
- One matmul sends it on to the previous layer, one turns it into the update for W

The backward pass computes two gradients



- The loss comes back as a gradient at the output
- One matmul sends it on to the previous layer, one turns it into the update for W

The backward pass computes two gradients



- The loss comes back as a gradient at the output
- One matmul sends it on to the previous layer, one turns it into the update for W
- Both are the same shape as the forward, so twice the cost

Where the 6 comes from

Two plus four is six

$$\text{FLOPs per token} = \underbrace{2N}_{\text{forward}}$$

Two plus four is six

$$\text{FLOPs per token} = \underbrace{2N}_{\text{forward}} + \underbrace{4N}_{\text{backward}}$$

Two plus four is six

$$\text{FLOPs per token} = \underbrace{2N}_{\text{forward}} + \underbrace{4N}_{\text{backward}} = 6N$$

Two plus four is six

$$\text{FLOPs per token} = \underbrace{2N}_{\text{forward}} + \underbrace{4N}_{\text{backward}} = 6N$$

- Over D training tokens, $C = 6ND$: the rule of thumb was a derivation all along

A transformer is a stack of identical layers

Tokens

A transformer is a stack of identical layers



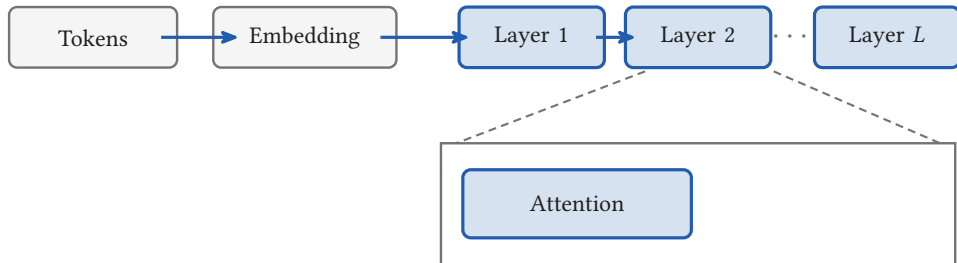
A transformer is a stack of identical layers



A transformer is a stack of identical layers

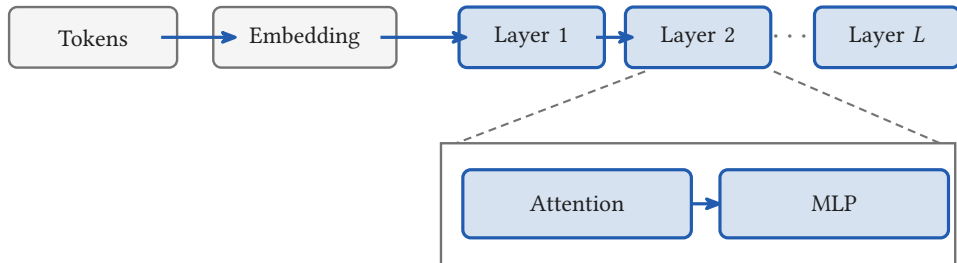


A transformer is a stack of identical layers



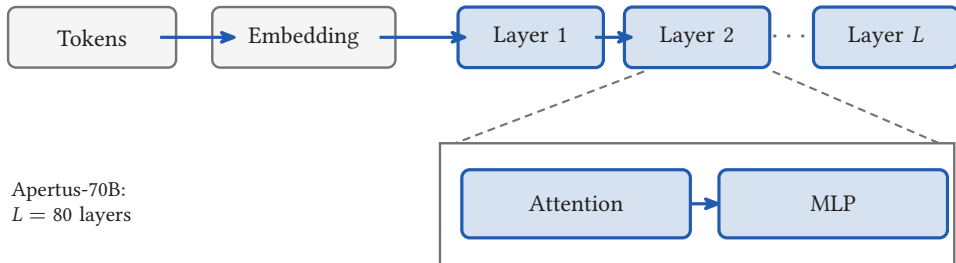
Every layer, the same two blocks and the same shapes

A transformer is a stack of identical layers



Every layer, the same two blocks and the same shapes

A transformer is a stack of identical layers



Apertus-70B:
 $L = 80$ layers

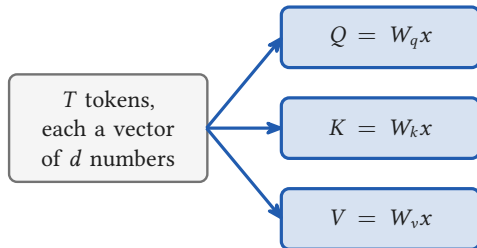
Every layer, the same two blocks and the same shapes

- Count one layer, multiply by L , and you have the model

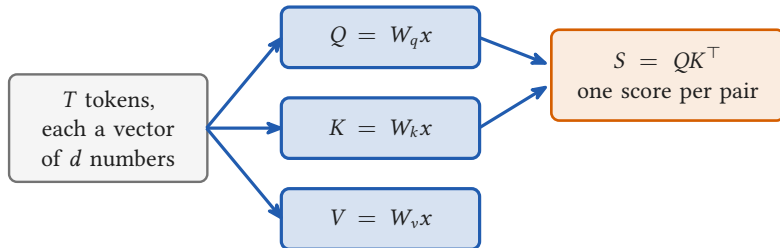
Attention, in one slide

T tokens,
each a vector
of d numbers

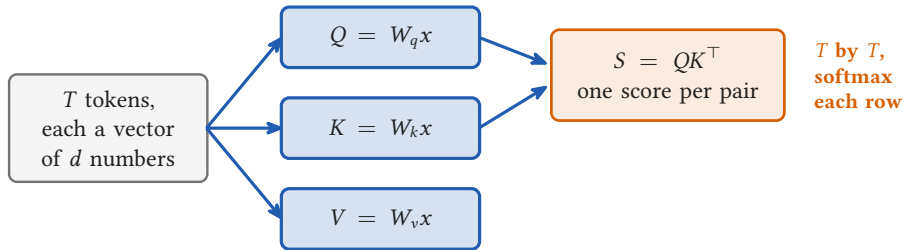
Attention, in one slide



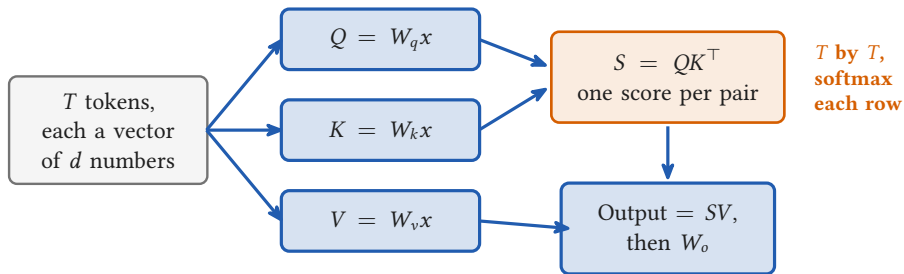
Attention, in one slide



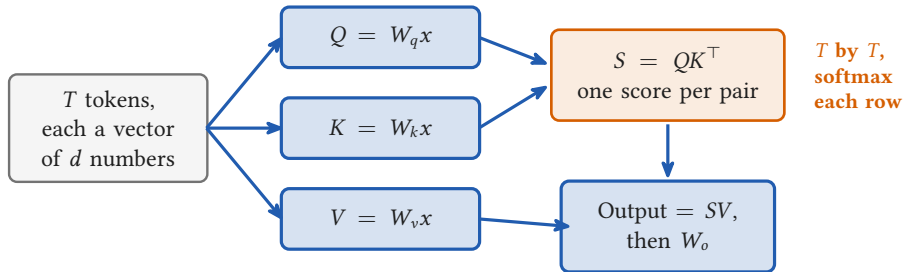
Attention, in one slide



Attention, in one slide

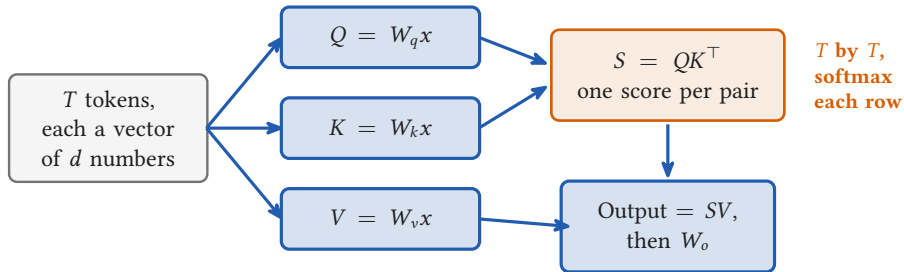


Attention, in one slide



- T is the context length: how many tokens the model reads at once

Attention, in one slide



- T is the context length: how many tokens the model reads at once
- Q , K , V and W_o are weight matrices. The scores are not

What the count leaves out

After the embedding, every token is a vector of d numbers. Typically d is a few thousand: Apertus-70B uses $d = 8192$.

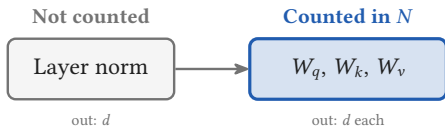
Not counted

Layer norm

out: d

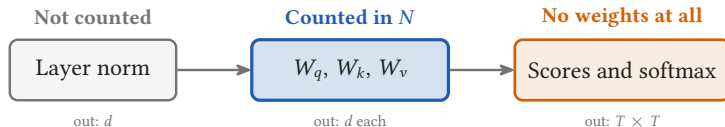
What the count leaves out

After the embedding, every token is a vector of d numbers. Typically d is a few thousand: Apertus-70B uses $d = 8192$.



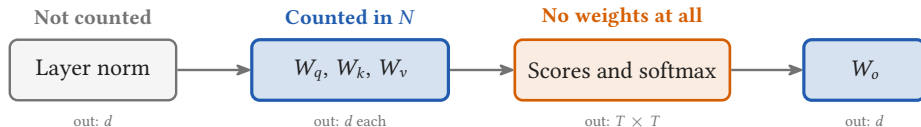
What the count leaves out

After the embedding, every token is a vector of d numbers. Typically d is a few thousand: Apertus-70B uses $d = 8192$.



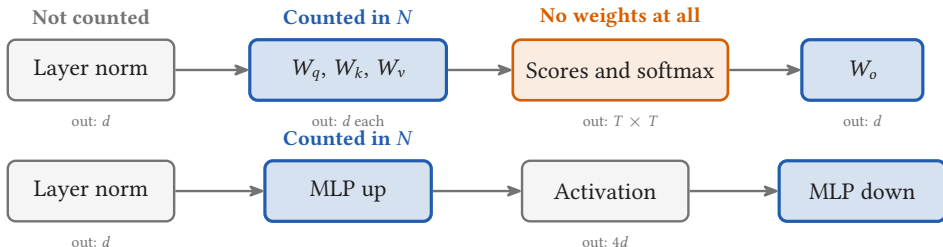
What the count leaves out

After the embedding, every token is a vector of d numbers. Typically d is a few thousand: Apertus-70B uses $d = 8192$.



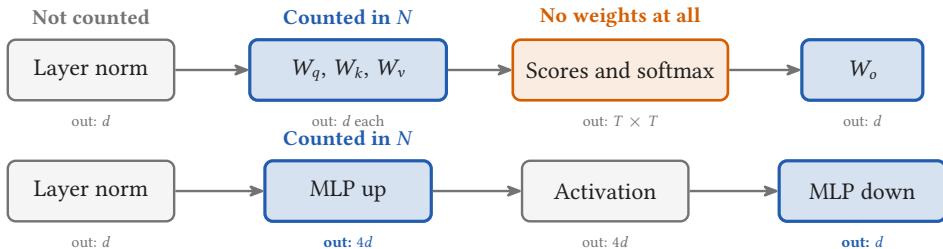
What the count leaves out

After the embedding, every token is a vector of d numbers. Typically d is a few thousand: Apertus-70B uses $d = 8192$.



What the count leaves out

After the embedding, every token is a vector of d numbers. Typically d is a few thousand: Apertus-70B uses $d = 8192$.



The MLP widens d to $4d$ and brings it back



Section 2

Counting one layer by hand

2

Counting one layer by hand

One layer at $d = 4$: 192 numbers



W_q

Counting one layer by hand

One layer at $d = 4$: 192 numbers



W_q



W_k



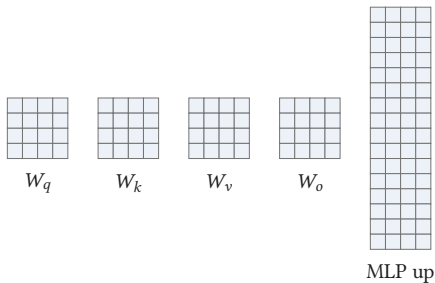
W_v



W_o

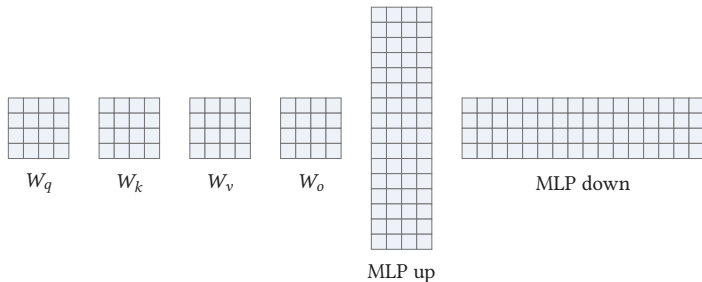
Counting one layer by hand

One layer at $d = 4$: 192 numbers



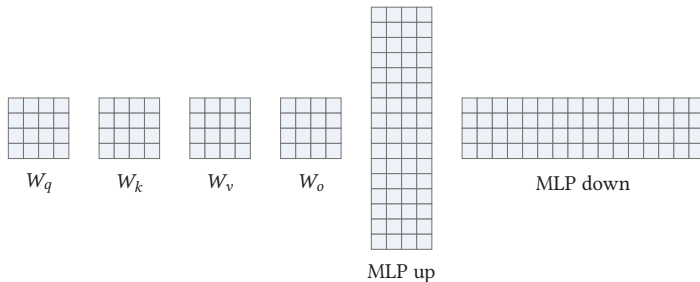
Counting one layer by hand

One layer at $d = 4$: 192 numbers



Counting one layer by hand

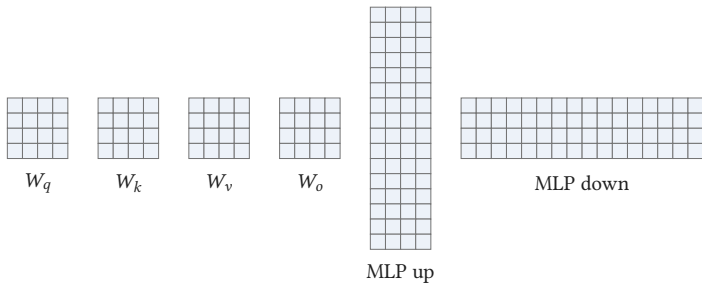
One layer at $d = 4$: 192 numbers



Attention: $64 = 4d^2$

Counting one layer by hand

One layer at $d = 4$: 192 numbers

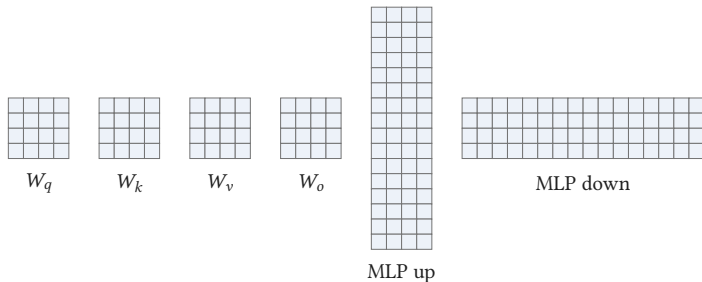


Attention: $64 = 4d^2$

MLP: $128 = 8d^2$

Counting one layer by hand

One layer at $d = 4$: 192 numbers



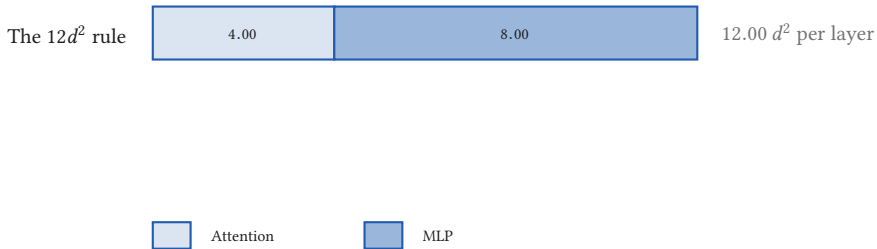
Attention: $64 = 4d^2$

MLP: $128 = 8d^2$

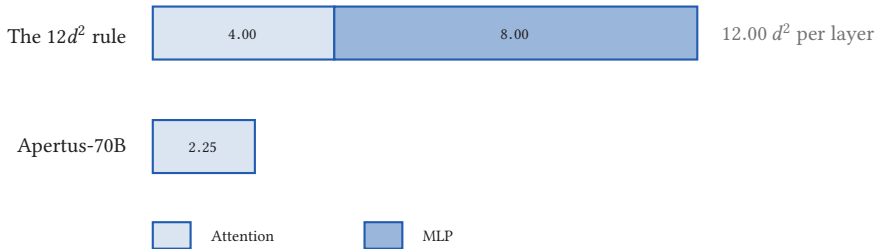
One layer: $192 = 12d^2$

The vanilla layer: full multi head attention, and a plain MLP that widens d to $4d$ and back. Every cell above is one parameter.

The same rule on a 2025 model is 6 percent low

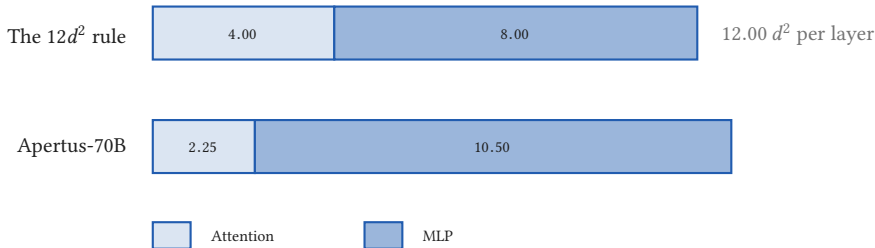


The same rule on a 2025 model is 6 percent low



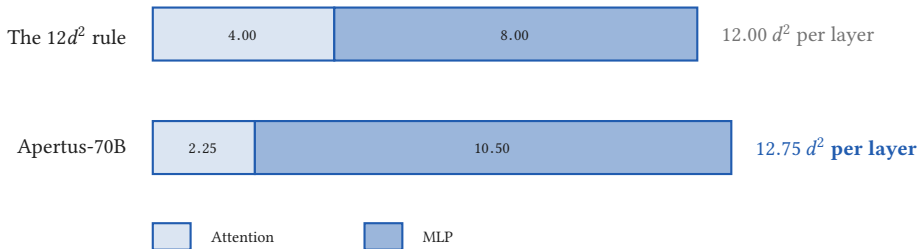
- Grouped attention: 8 key and value heads, not 64

The same rule on a 2025 model is 6 percent low



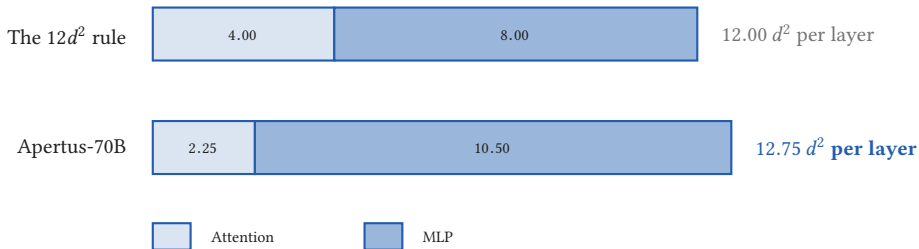
- Grouped attention: 8 key and value heads, not 64
- A wider MLP, 5.25 d instead of 4 d , and not gated

The same rule on a 2025 model is 6 percent low



- Grouped attention: 8 key and value heads, not 64
- A wider MLP, 5.25 d instead of 4 d , and not gated

The same rule on a 2025 model is 6 percent low



- Grouped attention: 8 key and value heads, not 64
- A wider MLP, 5.25 d instead of 4 d , and not gated
- $12d^2 \times 80$ layers gives 6.442×10^{10} against a published 6.845×10^{10}

Counting one layer by hand

Counting the real shapes: 70,599,843,840

80 layers, $d = 8192$, 64 heads, 8 key and value heads, head dim 128, hidden 43008, vocab 131072

W_q	8192×8192	67,108,864
W_k	8192×1024	8,388,608
W_v	8192×1024	8,388,608
W_o	8192×8192	67,108,864

Counting one layer by hand

Counting the real shapes: 70,599,843,840

80 layers, $d = 8192$, 64 heads, 8 key and value heads, head dim 128, hidden 43008, vocab 131072

W_q	8192×8192	67,108,864
W_k	8192×1024	8,388,608
W_v	8192×1024	8,388,608
W_o	8192×8192	67,108,864
Attention = $2.25 d^2$		150,994,944

Counting one layer by hand

Counting the real shapes: 70,599,843,840

80 layers, $d = 8192$, 64 heads, 8 key and value heads, head dim 128, hidden 43008, vocab 131072

W_q	8192×8192	67,108,864
W_k	8192×1024	8,388,608
W_v	8192×1024	8,388,608
W_o	8192×8192	67,108,864
Attention = $2.25 d^2$		150,994,944
MLP, two of 8192×43008		704,643,072

Counting one layer by hand

Counting the real shapes: 70,599,843,840

80 layers, $d = 8192$, 64 heads, 8 key and value heads, head dim 128, hidden 43008, vocab 131072

W_q	8192×8192	67,108,864
W_k	8192×1024	8,388,608
W_v	8192×1024	8,388,608
W_o	8192×8192	67,108,864
Attention = $2.25 d^2$		150,994,944
MLP, two of 8192×43008		704,643,072
One layer = $12.75 d^2$		855,638,016

Counting one layer by hand

Counting the real shapes: 70,599,843,840

80 layers, $d = 8192$, 64 heads, 8 key and value heads, head dim 128, hidden 43008, vocab 131072

W_q	8192×8192	67,108,864	Times 80 layers	68,451,041,280
W_k	8192×1024	8,388,608		
W_v	8192×1024	8,388,608		
W_o	8192×8192	67,108,864		
Attention = $2.25 d^2$		150,994,944		
MLP, two of 8192×43008		704,643,072		
One layer = $12.75 d^2$		855,638,016		

Counting one layer by hand

Counting the real shapes: 70,599,843,840

80 layers, $d = 8192$, 64 heads, 8 key and value heads, head dim 128, hidden 43008, vocab 131072

W_q	8192×8192	67,108,864	Times 80 layers	68,451,041,280
W_k	8192×1024	8,388,608	Embeddings, untied	2,147,483,648
W_v	8192×1024	8,388,608	Norm weights	1,318,912
W_o	8192×8192	67,108,864		
Attention = $2.25 d^2$		150,994,944		
MLP, two of 8192×43008		704,643,072		
One layer = $12.75 d^2$		855,638,016		

Counting one layer by hand

Counting the real shapes: 70,599,843,840

80 layers, $d = 8192$, 64 heads, 8 key and value heads, head dim 128, hidden 43008, vocab 131072

W_q	8192×8192	67,108,864
W_k	8192×1024	8,388,608
W_v	8192×1024	8,388,608
W_o	8192×8192	67,108,864
Attention = $2.25 d^2$		150,994,944
MLP, two of 8192×43008		704,643,072
One layer = $12.75 d^2$		855,638,016

Times 80 layers	68,451,041,280
Embeddings, untied	2,147,483,648
Norm weights	1,318,912
Total	70,599,843,840

Counting one layer by hand

Counting the real shapes: 70,599,843,840

80 layers, $d = 8192$, 64 heads, 8 key and value heads, head dim 128, hidden 43008, vocab 131072

W_q	8192×8192	67,108,864	Times 80 layers	68,451,041,280
W_k	8192×1024	8,388,608	Embeddings, untied	2,147,483,648
W_v	8192×1024	8,388,608	Norm weights	1,318,912
W_o	8192×8192	67,108,864		
Attention	$= 2.25 d^2$	150,994,944	Total	70,599,843,840
MLP, two of	8192×43008	704,643,072	Published: 70.60 billion	
One layer	$= 12.75 d^2$	855,638,016		

Counted from [swiss-ai/Apertus-70B-2509/config.json](https://github.com/swiss-ai/Apertus-70B-2509/blob/main/config.json). Nothing here is asserted.



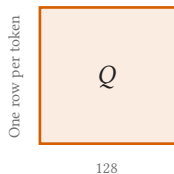
Section 3

Not all FLOPs are equal

3

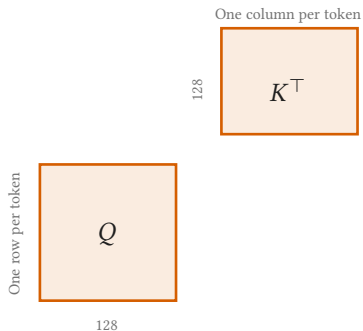
Not all FLOPs are equal

The attention scores have no weights

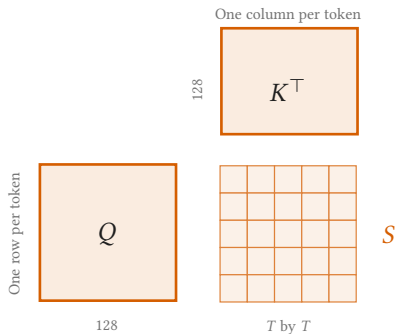


Not all FLOPs are equal

The attention scores have no weights



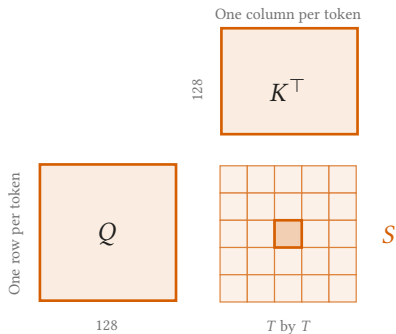
The attention scores have no weights



- One score per pair of tokens, so the grid grows with the context

Not all FLOPs are equal

The attention scores have no weights

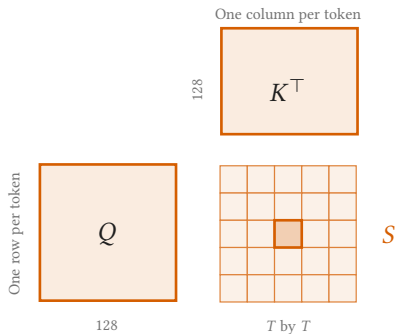


Each cell: 128 multiply accumulates

- One score per pair of tokens, so the grid grows with the context

Not all FLOPs are equal

The attention scores have no weights

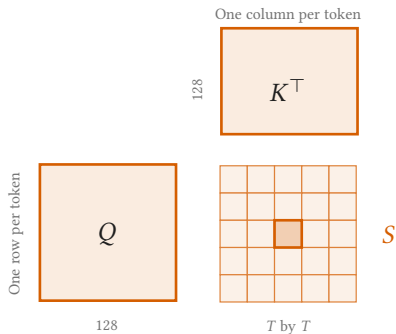


Each cell: 128 multiply accumulates

- One score per pair of tokens, so the grid grows with the context
- No weight matrix anywhere in it, so N never sees it

Not all FLOPs are equal

The attention scores have no weights



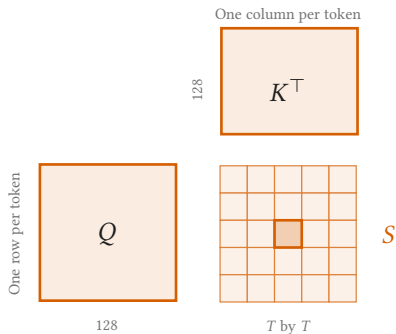
Each cell: 128 multiply accumulates

- One score per pair of tokens, so the grid grows with the context
- No weight matrix anywhere in it, so N never sees it

$\underbrace{2dT}_{\text{scores}}$

Not all FLOPs are equal

The attention scores have no weights



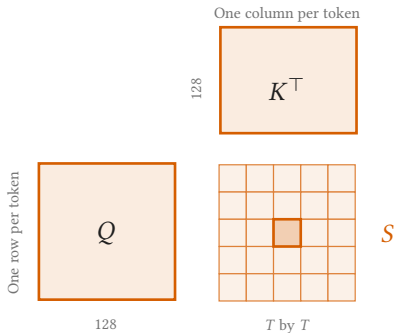
Each cell: 128 multiply accumulates

- One score per pair of tokens, so the grid grows with the context
- No weight matrix anywhere in it, so N never sees it

$$\underbrace{2dT}_{\text{scores}} + \underbrace{2dT}_{\text{values}}$$

Not all FLOPs are equal

The attention scores have no weights



Each cell: 128 multiply accumulates

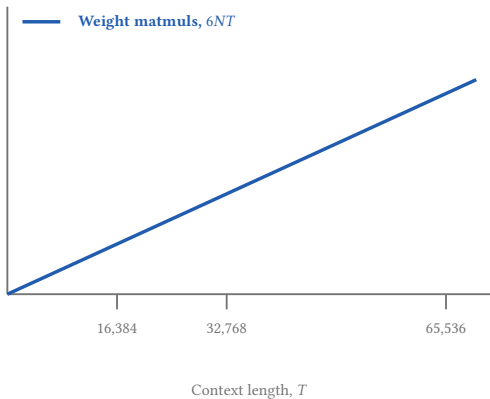
- One score per pair of tokens, so the grid grows with the context
- No weight matrix anywhere in it, so N never sees it

$$\underbrace{4dT}_{\text{forward}} + \underbrace{8dT}_{\text{backward}} = 12 d T$$

- Per token, per layer, so $12 L d T$ for the whole model

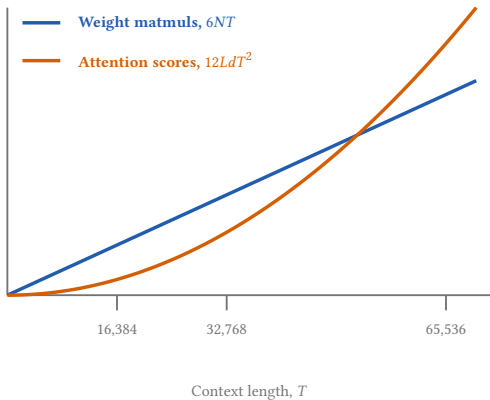
Not all FLOPs are equal

Where the two costs cross



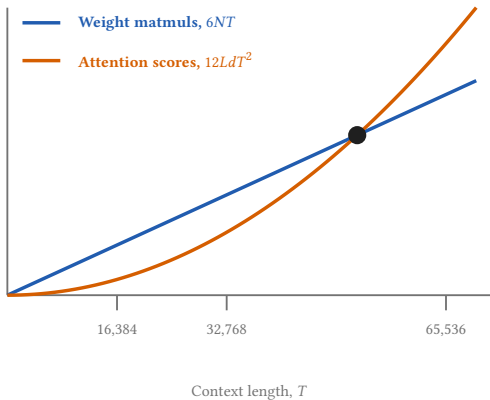
Not all FLOPs are equal

Where the two costs cross



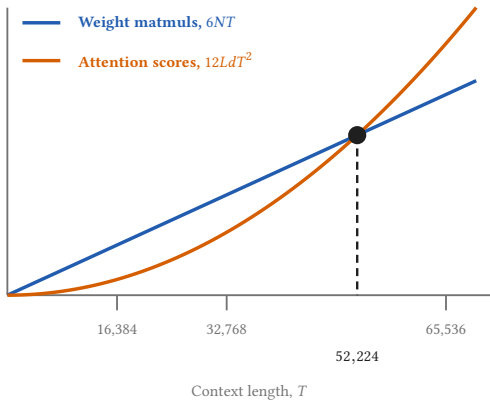
Not all FLOPs are equal

Where the two costs cross



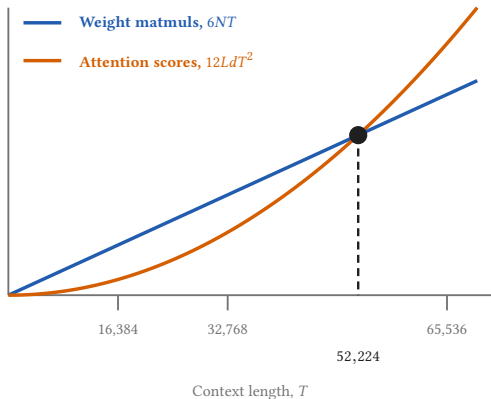
Not all FLOPs are equal

Where the two costs cross



Not all FLOPs are equal

Where the two costs cross

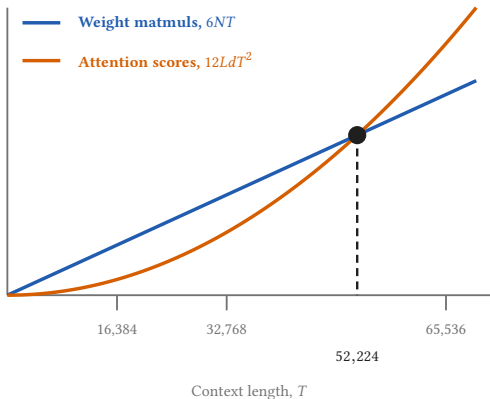


$$6N = 12 L d T^*$$

- Set the two terms equal

Not all FLOPs are equal

Where the two costs cross

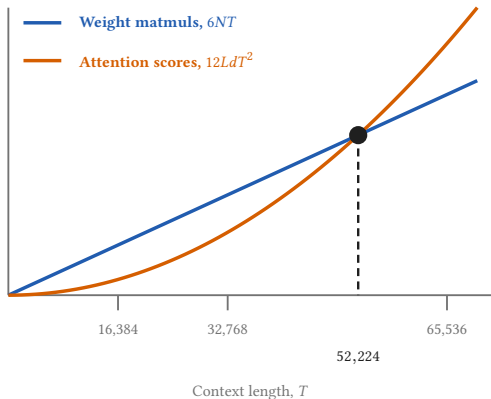


$$T^* = \frac{N}{2Ld}$$

- Set the two terms equal
- Divide through: what one layer holds is what is left

Not all FLOPs are equal

Where the two costs cross

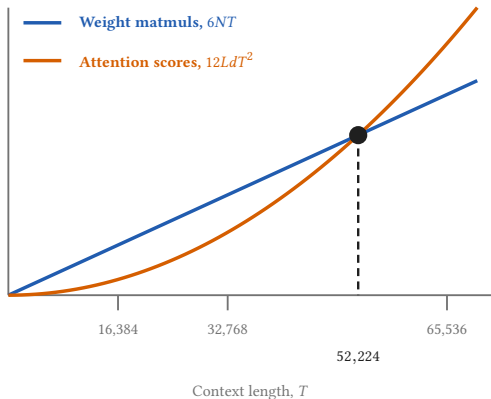


$$T^* = \frac{P_{\text{layer}}}{2d}$$

- Set the two terms equal
- Divide through: what one layer holds is what is left
- That layer holds $12.75 d^2$, counted a moment ago

Not all FLOPs are equal

Where the two costs cross



$$T^* = \frac{P_{\text{layer}}}{2d} = 6.375 d = 52,224$$

- Set the two terms equal
- Divide through: what one layer holds is what is left
- That layer holds $12.75 d^2$, counted a moment ago
- Not a constant of transformers: a vanilla layer crosses at $6d$, a gated one at $8d$

Both attention matmuls counted, no credit taken for causal masking.

Not all FLOPs are equal

What attention adds on top of 6ND

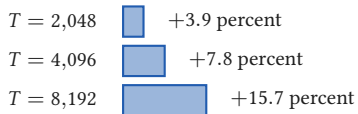
$T = 2,048$  +3.9 percent

What attention adds on top of 6ND

$T = 2,048$		+3.9 percent
$T = 4,096$		+7.8 percent

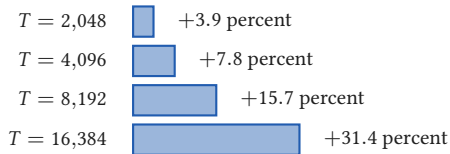
- 4,096 is the sequence length Apertus pretrained at

What attention adds on top of 6ND



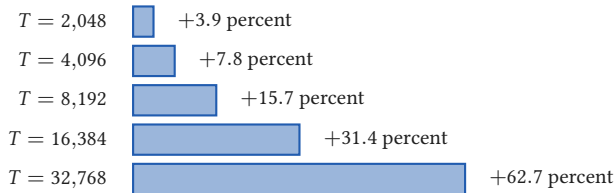
- 4,096 is the sequence length Apertus pretrained at

What attention adds on top of 6ND



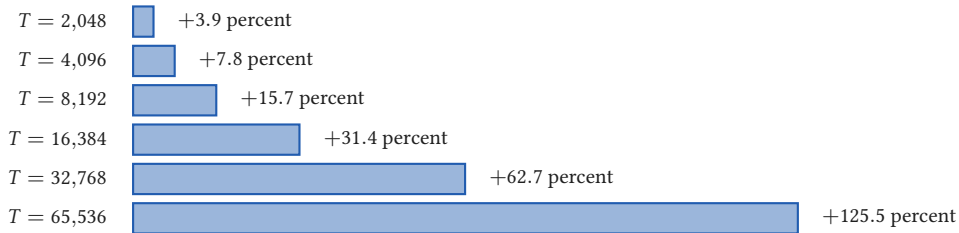
- 4,096 is the sequence length Apertus pretrained at

What attention adds on top of 6ND



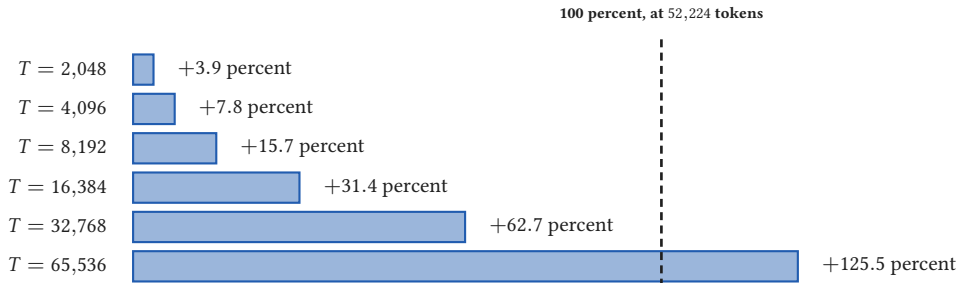
- 4,096 is the sequence length Apertus pretrained at

What attention adds on top of 6ND



- 4,096 is the sequence length Apertus pretrained at
- 65,536 is the model's own maximum context, where attention more than doubles the bill

What attention adds on top of 6ND



- 4,096 is the sequence length Apertus pretrained at
- 65,536 is the model's own maximum context, where attention more than doubles the bill
- Above the crossover, 6ND is the smaller of the two terms